

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI – 59001

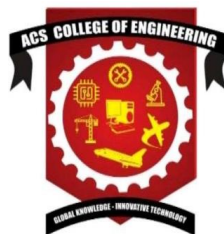


**PROJECT REPORT
ON
“WEED IDENTIFICATION USING DEEP LEARNING AND IMAGE
PROCESSING”**

Submitted in partial fulfillment for the requirement of 8th semester

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING
SUBMITTED BY
ABHILASH MADHAV M (1AH19CS002)

UNDER THE GUIDANCE OF
Ms GANGA B M
Assistant Professor

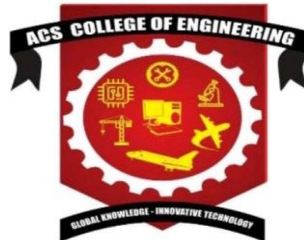


DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ACS COLLEGE OF ENGINEERING
KAMBIPURA, MYSORE ROAD, BENGALURU – 560074
2022-2023

Department of Computer Science and Engineering

ACS COLLEGE OF ENGINEERING

#74, Kambipura, Mysore road, Bengaluru – 560074



CERTIFICATE

Certified that the Project on topic " **WEED IDENTIFICATION USING DEEP LEARNING AND IMAGE PROCESSING** " Project of 8th Semester has been successfully submitted by **ABHILASH MADHAV M(1AH19CS002)** Bonafide students at **ACS COLLEGE OF ENGINEERING** affiliated to Visveswaraya Technological University, Belgaum during the year 2022-2023. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report submitted to the department. The Project report has been approved as it satisfies academic requirements in respect of project work as prescribed in the 8th semester.

Signature of the guide

Signature of the Coordinator

Signature of HOD

Name of the Examiner

Signature with Date

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without mentioning names of the people who made it possible, whose constant guidance and encouragement crowned our effort with success.

We are thanking full to our honorable chairman **Sri. A. C. Shanmugam** for providing us with the better facilities and his encouragement also helped us in completion of project.

We are grateful tour institution, **A.C.S College of Engineering** with its ideals and inspirations for having provided us with the facilities, which made this, project a success.

We earnestly thank **Dr. M. S. Murali, Principal, ACSCE**, for facilitating academic excellence in the college that helped us in completing this project.

We wish to extend our profound thanks to **Dr. T Senthil Kumaran**, Dean and Head of the department, Computer Science and Engineering, for giving us the consent to carry out this project.

We would like to express our sincere thanks to the project coordinator and guide, **Ms. Ganga B M**, Assistant professor, Department of CSE, for her able guidance and valuable advice at every stage of our project, which helped us in the successful completion of the project.

We convey our thanks to friends for all the help they provided. Last but not the least we would like thank all the staff members and the institute, in general, for extending a helping hand and making this possible.

ABHILASH MADHAV M

(1AH19CS002)

ABSTRACT

Weeds are plants filling in an off-base spot that contend with crops for water, light, supplements and space, causing decrease in yield and compelling utilization of hardware and cause a disturbance in agriculture. Weed picking is one of the most laborious jobs in the fields. Weeds will host disease and illnesses that can spread to developed yields. We are proposing a solution in which the device is integrated with camera and there will be a live video streaming in which it will detect the weed in the crop by using image processing. This system will distinguish the crop and weed. Our system will use a Convolution neural network algorithm to extract the features from images and train them by using a neural network.

TABLE OF CONTENTS

ABSTRACT	i
ACKNOWLEDGEMENT	ii
CONTENTS	iii
LIST OF FIGURES	iv
CHAPTER 1: INTRODUCTION	1-3
General Introduction	1
1.1 Existing System.....	2
1.2 Proposed System.....	2
1.3 Problem Statement	3
1.4 Objectives	3
CHAPTER 2: LITERATURE SURVEY	4-6
2.1 RNN- and CNN-based weed detection for crop improvement	4
2.2 Weed detection using convolution neural network	4
2.3 A Survey on Weed Detection using Image Processing	5
2.4 Weed and crop segmentation.....	5
2.5 Detection of Weed using Neural Networks.....	6
2.6 Real-Time Weed Detection using Machine Learning and Stereo-Vision	6
CHAPTER 3: SYSTEM REQUIREMENTS AND DESIGN	7-8
3.1 System Analysis.....	7
3.2 Functional Requirements.....	7
3.3 Non-Functional Requirements.....	8
3.4 Tools And Technologies.....	8
CHAPTER 4: SYSTEM DESIGN.....	09-18
4.1 System Architecture	09
4.2 Input And Output Design	10
4.3 Object Oriented Design.....	11
4.3.1 Data Flow Diagram	12
4.3.2 Use Case Diagram.....	12
4.3.3 Class Diagram.....	13

4.3.4 Sequence Diagram.....	13
4.3.5 Activity Diagram	14
CHAPTER 5: IMPLEMENTATION.....	18-29
5.1 Steps For Implementation	18
5.2 Module Description.....	24
5.2.1 Pre – Processing	25
5.2.2 Image Enhancement	25
5.2.3 Image Segmentation	26
5.2.4 Feature Extraction	26
5.2.5 Classification	26
5.3 Coding	29
CHAPTER 6: TESTING	42-49
6.1 Testing	42
6.2 White Box.....	42
6.3 Black Box	43
6.4 Unit Testing	45
6.5 Integration Testing	46
6.6 System Testing.....	48
CHAPTER 7: RESULTS AND DISCUSSIONS.....	49
CONCLUSION	51
FUTURE ENCHANCEMENT	52
REFERENCES	53

LIST OF FIGURES

Figure no	Figure Name	Page no
4.1	Deep-Convolutional Neural Network Architecture	09
4.2	Convolutional Neural Network General Architecture	11
4.3	Data Flow Diagram	12
4.4	Use Case Diagram	12
4.5	Class Diagram	13
4.6	Sequence Diagram	13
4.7	Activity Diagram	14
4.8	Data Flow Diagram for Pre-processing	14
4.9	Data Flow Diagram for Identification	15
4.9.1	Data Flow Diagram for Feature Extraction	15
4.9.2	Data Flow Diagram for Classification and Detection	16
4.9.3	Use Case Diagram for weed detection	17
5.1	Data flow Diagram of Training and Testing Phase	24
5.2	Convolutional Neural Network Layers	27
5.3	Structure of CNN in matrix format	28
5.4	Graph of positive ReLU Func	29
6.1	White box Testing	43
6.2	Black Box testing	44
7.1	Implementation of source code	49
7.2	Weed detection with its impact factor	49
7.3	Dataset collected	50

CHAPTER 1

INTRODUCTION

As weeds are frequently distributed non-uniformly within a field, weeding is a very hard work. Every year a large amount of herbicide is used for removing weeds from agricultural fields which is not only expensive but also a source of environmental pollution. Moreover, both costly and time consuming is caused due to hand labor. Therefore, weed control is a necessary management practice in agricultural systems, which is critical to sustain crop productivity and quality. Farmers need alternatives for weed control due to the desire to reduce chemical use and production costs. For instance, when growing vegetables such as carrots, onions and cabbage the weeds can be removed by special harrows but it cannot support for economic system. Nowadays, uniform spraying is the most common method for herbicides application. However, this method is inefficient and cost-ineffective as weed distribution is usually non-uniform and highly aggregated in clumps within the arable field. There could be many parts of the field that have none or insignificant volume of weeds.

This property of the weed distributions makes the development of site-specific management feasible. In this management, the number of herbicides applied is reduced through spraying only the weed infested area of a field, where different selective herbicides with corresponding application rates are applied to control weed differently. The machine vision-based approach uses shape, texture, color and location based features individually or jointly to discriminate between weed and crop. In the other research, Caltrans sprays roadside plant material with herbicide to prevent the weeds from becoming a fire hazard during the summer. The first step in identifying weeds within an image involves classifying the pixels. The pixels shall be classified using a point operation. The surrounding pixels will not bias a pixel's classification. The purpose of segmenting the image into plant and background pixels is to detect the amount of plant material within a specific area.

If the amount of plant material reaches a specific threshold, the area is targeted for herbicidal spray application. The spray threshold is set too close to the background misclassification rate, then herbicide will be wasted spraying background. Therefore, a larger misclassification rate limits the smallest plant that can be detected without targeting the background for spray. A system that could make use of the spatial distribution information in real-time and apply only the necessary amounts of herbicide to the weed-infested area would be much more efficient and minimize environmental damage. Therefore, a high spatial resolution, real-time weed

infestation detection system seems to be the solution for site-specific weed management. In this paper, three main parts presented for weed and crop classification. It has got

- (1) Image acquisition and gray transformation,
- (2) Image segmentation and noise removal, and
- (3) Classification of weed and crop.

Two classification methods are studied in this research. They are classification based on intensity method and area thresholding classification method. The first method is depended on the intensity of the images. However, this algorithm cannot operate to classify weed and crop automatically. So, the area thresholding method is used to get reliable classified images. These methods are executed depending on size of the plants. By using digital camera, the input image is obtained. Most of images are needed to change from the color images into grayscale images for easy and fast processing. The segmentation step implemented by using thresholding method. In the classification part, the areas of segmented image are then compared with the selected threshold for classification of weeds and crop.

1.1 Existing system

After a literature survey, a rural undertaking of self-governing weed control has been isolated into three fundamental issues. They are restriction and route of the machine or carry out, location and order of plants and weed control techniques. The sub-assignments with the strong layout are the ones that this exploration focuses on. Albeit the issue of building a working self-governing weed controlling gadget isn't restricted to the three issues introduced, one should in any case tackle at any rate one sub-task from every one of the three issues for progress.

1.2 Proposed system

The approach proposed in this research for the identification of weeds is composed of two stages. The first stage consists of the state-of-art Center Net algorithm [21] for detecting Bok choy in this study. Images of book choy are collected and used as input data for training the neural network. The trained neural network is used for detecting Bok choy and drawing bounding boxes around them, which generates bounding box coordinates and associate class probabilities as the output. In the second stage, a color index-based segmentation is performed on vegetation (pixels) outside the bounding boxes using color information, returning visual classification regarding the presence of weeds in the image. The employed color index was determined and evaluated through Genetic Algorithms (GAs) according to Bayesian classification error. Procedural steps of the proposed method are shown in Fig. 1. Rest of this

section provides details of each step. Center Net model training and testing were performed in the PyTorch deep learning environment using a graphic processing unit (NVIDIA GeForce RTX 2080 SUPER, NVIDIA; Santa Clara, USA). Genetic algorithm was developed and implemented using the Python language with OpenCV library. Both algorithms were operated from a computer with an Intel(R) Core (TM) i9-9900K CPU @ 3.60GHz central processing unit (CPU).

1.3 Problem statement

Deep learning is a modern technique for image processing and data analysis with promising results and great potential. Successfully applied in various fields, it has recently entered the field of agriculture to address such agricultural problems as disease identification, fruit/plant classification, fruit counting, pest identification, and weed detection. The latter was the subject of our work. Weeds are harmful plants that grow in crops, competing for things like sunlight and water and causing crop yield losses. Traditional data processing techniques have several limitations and consume a lot of time.

1.4 Objectives

The aim of this work is to develop a deep convolutional neural network that learned features directly from the input of weed images and classifying the weed into crop or weed and if it is a weed, it will show the corresponding weed name. Therefore, objectives of proposed system are as follows:

- 1) To develop a deep convolutional neural network that can identify weeds and classify them into different classes.
- 2) To develop a network that learns from datasets and still demonstrates superior performance for weed detection.
- 3) To develop an efficient approach to training a deep learning model with a balanced dataset.
- 4) To do the performance analysis of the proposed system for validation

CHAPTER 2

LITERATURE SURVEY

2.1 Title: RNN- and CNN-based weed detection for crop

improvement Author: Brahim Jabir* , Loubna Rabhi , Nouredine Falih

Publication: 2021

Abstract: Deep learning is a modern technique for image processing and data analysis with promising results and great potential. Successfully applied in various fields, it has recently entered the field of agriculture to address such agricultural problems as disease identification, fruit/plant classification, fruit counting, pest identification, and weed detection. The latter was the subject of our work. Weeds are harmful plants that grow in crops, competing for things like sunlight and water and causing crop yield losses. Traditional data processing techniques have several limitations and consume a lot of time. Therefore, we aimed to take inventory of deep learning networks used in agriculture and conduct experiments to reveal the most efficient ones for weed control.

2.2 Title: WEED DETECTION USING CONVOLUTION NEURAL NETWORK

Author: Arvinth S*1, Balakrishnan A*2, Harikrishnan M*3, Jeydheepan J*4

Publication: 2021

Abstract: Weeds are the plants filling in an off-base spot which contend with crop for water, light, supplements and space, causing decrease in yield and compelling utilization of hardware and cause a disturbance in agriculture. Weed picking is one of the laborious job in fields. Weeds will host disease and illnesses that can spread to developed yields. We are proposing a solution in the device is integrated with camera and there will be a live video streaming in that it will detect the weed in the crop by using image processing. This system will distinguish the crop and weed. Our system will use a Convolution neural network algorithm to extract the features from image and train them by using neural network.

2.3 Title: A Survey on Weed Detection using Image Processing

Author: Basavarajeshwari, Prof. S. P. Madhavanavar

Publication: 2017

Abstract: Agriculture is the origins of human sustenance in this world. The detection and classification of weeds are of most important technical and economical importance in the agricultural industry. In past days weed detection was done by employing some men, particularly for that intention. In olden days, weed detection was done by inspecting each and every place in the field. Then weeds were manually removed. Later with the improvement in the technology, people started using herbicides to take out the weeds. But to identify the weed still physical power was used in many parts of the world. Later there came few methods to discover the weeds without human intervention but due to lack of their accuracy, they were incapable to reach to the public. Then image processing was used for this purpose. In future it is planned to detect the weed in the crop by using image processing techniques. This paper focuses on survey of weed detection using image processing techniques.

2.4 Title: WEED AND CROP SEGMENTATION AND CLASSIFICATION USING AREA THRESHOLDING

Author: Su Hnin Hlaing¹ , Aung Soe Khaing

Publication: 2014

Abstract: In the agricultural industry, the weed and crop identification and classification are major technical and economical importance. Two classification algorithms are focused in this paper. And the better classification algorithm has been selected to classify weed and crop from the images. There are three main parts of proposed system are segmentation, classification and error calculation. The developed algorithm based on area thresholding has been tested on weeds and various locations. Forty one sample images have been tested and the result of some weed coverage rate is illustrated. Moreover, the misclassification rate is also computed. An algorithm has been done to automate the tasks of segmentation and classification. The overall process is implemented in MATLAB.

2.5 Title: Detection of Weed using Neural Networks

Author: Mrs. R. Dhayabarani , Aravinth K

Publication: 2018

Abstract: Improving the efficiency of the agriculture fields will increase the food resources in the response of the huge population in the world. In the fields, we are facing various difficulties such as weeds and plant diseases. We should detect the weeds and remove it. Nowadays we do not get more man power to work in the fields. Implementing machine learning methods such as convolutional neural networks on agriculture has gained immense attention in recent years. We are using convolutional neural networks to classify the plants. Automatic plant type identification process could offer a great help for application of pesticides, fertilization and harvesting of different species on-time in order to improve the production processes of food and drug industries. It could reduce the labor cost. Following the preprocessing step, Convolutional Neural Network architecture is employed to extract the features of images.

2.6 Title: WEED DETECTION USING IMAGE PROCESSING

Author: Ajinkya Paikekari¹, Vrushali Ghule², Rani Meshram³, V.B. Raskar⁴

Publication: 2016

Abstract: Agriculture is one of the origins of human sustenance in this world. Nowadays due to growing population we need the greater productive capability of the agriculture to meet the demands. In olden days, people used natural methods to increase the productivity, such as using the cow dung as a fertilizer in the fields. That resulted in an increase in the productivity enough to meet the requirements of the population. But later people thought of earning more profits by getting more outcome. So, there came a revolution called “Green Revolution”. After this period usage of deadly poisons as herbicides has increased to a drastic level. By doing so we got success in increasing the productivity but we have forgotten the damage done to the environment, which will raise a doubt in our sustenance on this beautiful earth. So, in this project, we have implemented some methods to reduce the usage of herbicides by spraying them only in the areas where weed is present. In this paper, we have implemented image processing using MATLAB to detect the weed areas in an image we took from the fields.

CHAPTER 3

SYSTEM REQUIREMENT AND SPECIFICATION

3.1 Functional Requirements

Useful requirements describe the product's internal activities: that is, the technical subtleties, monitoring and handling of data and other specific functionality demonstrating how to satisfy the use cases. They are upheld by non-utilitarian prerequisites that force the plan or execution of imperatives.

- System should process the data.
- System should segment the weed images.
- System should detect the weed image.
- System should predict weed or crop images using plant images.

3.2 Non-Functional Requirements

Unnecessary prerequisites are requirements that suggest parameters that can be used to assess a framework's operation rather than specific activities. This should be distinguished from useful necessities indicating explicit behavior or capabilities. Reliability, flexibility, and price are common non-practical necessities.

Non-practical preconditions are often referred to as system utilities. Different terms for non-practical necessities are "limitations, "quality characteristics" and "prerequisites for administration".

The architecture should be created in order to incorporate new modules and functionalities, thereby promoting application development. The cost should be small as a result of programming packages being freely accessible.

- Usability : System Should be userFriendly
- Reliability : System should beReliable
- Performance : System Should not take excess time in detecting the coffee leaf disease

- Supportability : System should be easily updatable for future enhancement

➤ **Hardware requirements**

- System : Pentium IV 2.4 GHz/intel i3/i4.
- Hard Disk : 40GB.
- Monitor : 15 VGAColor.
- RAM : 512 MbMinimum

➤ **Software requirements**

- Operating system : Windows XP/ Windows 7 or More
- Software Packages : Tensorflow , OpenCv
- Coding Language: Python.
- Toolbox : Image processing toolbox.

CHAPTER 4

SYSTEM DESIGN

Design Overview

Design overview explains the architecture that would be used for developing a software product. It is an overview of an entire system, identifying the main components that would be developed for the product and their interfaces.

4.1 System Architecture

A Deep-CNN is type of a DNN consists of multiple hidden layers such as convolutional layer, RELU layer, Pooling layer and fully connected a normalized layer. CNN shares weights in the convolutional layer reducing the memory footprint and increases the performance of the network. The important features of CNN lie with the 3D volumes of neurons, local connectivity and shared weights. A feature map is produced by convolution layer through the convolution of different sub-regions of the input image with a learned kernel. Then, a non-linear activation function is applied through ReLu layer to improve the convergence properties when the error is low. In pooling layer, a region of the image/feature map is chosen and the pixel with the maximum value among them or average values is chosen as the representative.

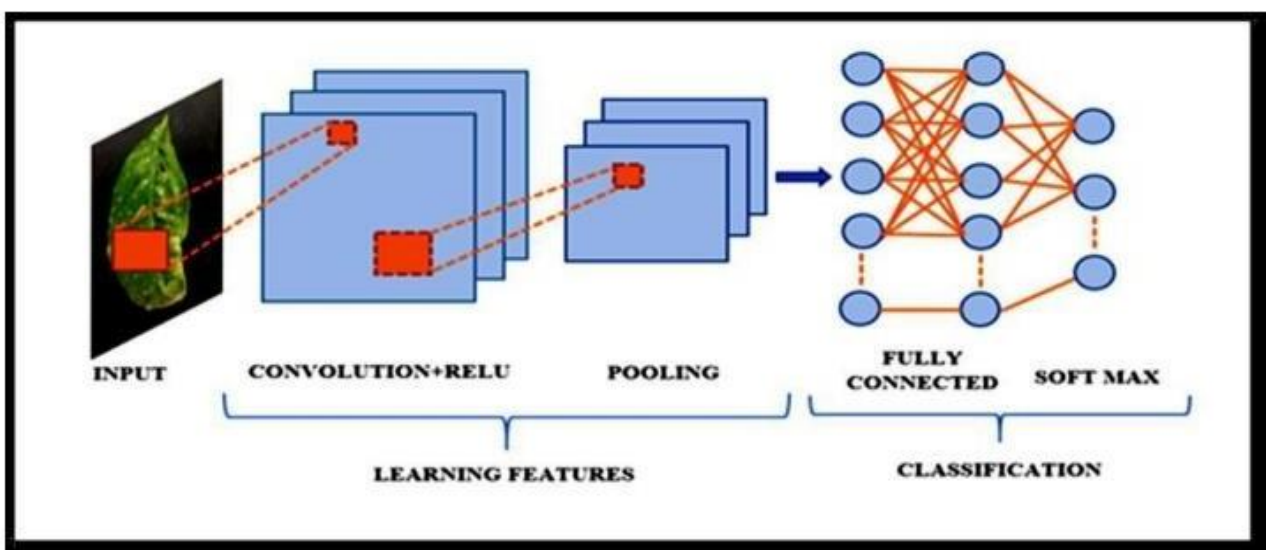


Fig 4.1 Deep-Convolutional Neural Network Architecture

This results a large reduction in the sample size. Sometimes, traditional Fully- Connected (FC) layer will be used in conjunction with the convolutional layers towards the output stage. In CNN architecture, usually convolution layer and pool layer are used in some combination. The pooling layer usually carries out two types of operations viz. max pooling and means pooling. In mean pooling, the average neighborhood is calculated within the feature points and in max pooling it is calculated within a maximum of feature points. Mean pooling reduces the error caused by the neighborhood size limitation and retains background information. Max pooling reduces the convolution layer parameter estimated error caused by the mean deviation and hence retains more texture information.

4.2 Input and Output Design

The proposed method follows these stages:

- **Data Set:**

The dataset for training is obtained from weed Image Database Consortium (LIDC) and Image Database Resource Initiative (IDRI). LIDC and IDRI consist of 1000 weed images of both large and small tumors saved in Digital Imaging and Communications in Medicine (DICOM) format.

- **Image Segmentation:**

The segmentation of photographs is the phase where the visual image is partitioned into several parts. This normally helps to identify artifacts and boundaries. The aim of segmentation is to simplify the transition in the interpretation of a picture into the concrete picture that can be clearly interpreted and quickly analyzed.

- **Pre-Processing:**

In preprocessing stage, the median filter is used to restore the image under test by minimizing the effects of the degradations during acquisition. Various preprocessing and segmentation techniques of lung nodules are discussed in. The median filter simply replaces each pixel value with the median value of its neighbors including itself. Hence, the pixel values which are very different from their neighbors will be eliminated.

- **Convolutional Neural Networks:**

A CNN is type of a DNN consists of multiple hidden layers such as convolutional layer, RELU layer, Pooling layer and fully connected a normalized layer. CNN shares weights in the convolutional layer reducing the memory footprint and increases the performance of the network. The important features of CNN lie with the 3D volumes of neurons, local connectivity and shared weights. A feature map is produced by convolution layer through convolution of

different sub regions of the input image with a learned kernel. Then, a non-linear activation function is applied through ReLU layer to improve the convergence properties when the error is low. In pooling layer, a region of the image/feature map is chosen and the pixel with maximum value among them or average values is chosen as the representative pixel so that a 2x2 or 3x3 grid will be reduced to a single scalar value. This results in a large reduction in the sample size. Sometimes, traditional Fully-Connected (FC) layer will be used in conjunction with the convolutional layers towards the output stage.

4.3 Object oriented Design

A CNN is composed of several kinds of layers:

- **Convolutional layer:** creates a feature map to predict the class probabilities for each feature by applying a filter that scans the whole image, few pixels at a time.
- **Pooling layer (down-sampling):** scales down the amount of information the convolutional layer generated for each feature and maintains the most essential information (the process of the convolutional and pooling layers usually repeats several times).
- **Fully connected input layer:** flattens the outputs generated by previous layers to turn them into a single vector that can be used as an input for the next layer.
- **Fully connected layer:** Applies weights over the input generated by the feature analysis to predict an accurate label.

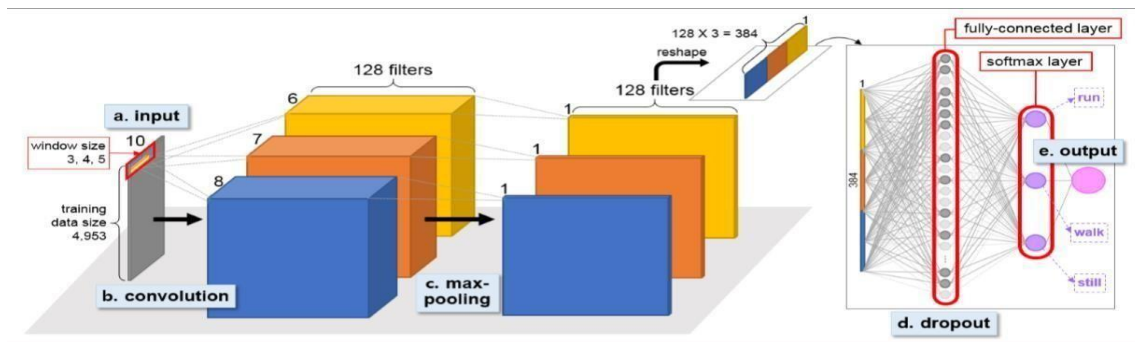


FIG 4.2 Convolutional Neural Network General Architecture

4.3.1 Data flow diagrams

A dataflow outline is a tool for referring to knowledge progression from one module to the next module as shown in Fig 4.3 This graph gives the data of each module's info and yield. The map has no power flow and there are no circles at the same time.

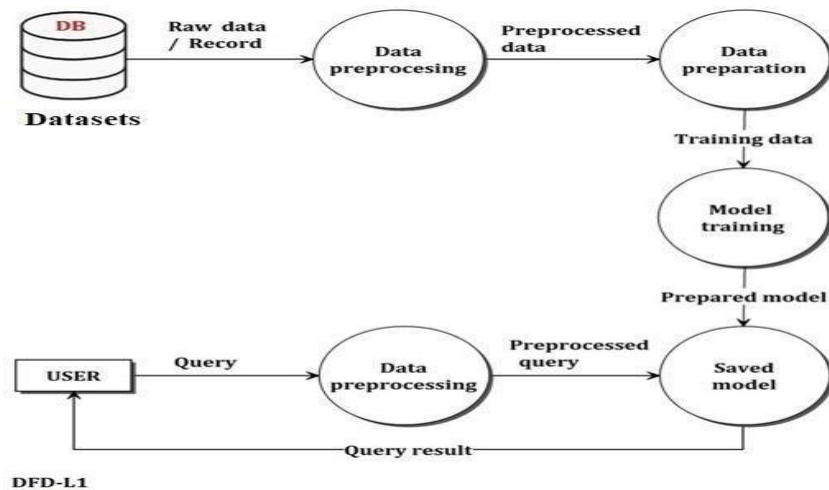


Fig 4.3 Data flow diagrams

4.3.2 Use Case Diagram

Use case diagram is the boundary, which defines the system of interest in relation to the world around it. The actors, usually individuals involved with the system defined according to their roles. The use cases are the specific roles played by the actors within and around the system.

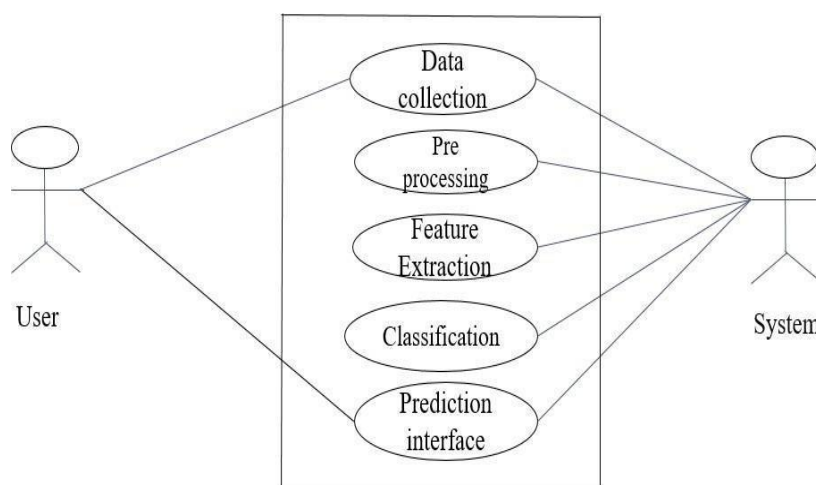


FIG 4.4 Use Case Diagram

4.3.4 Class Diagrams

Class diagrams are the main building block in object-oriented modeling. They are used to show the different objects in a system, their attributes, their operations and the relationships among them as shown in the Fig

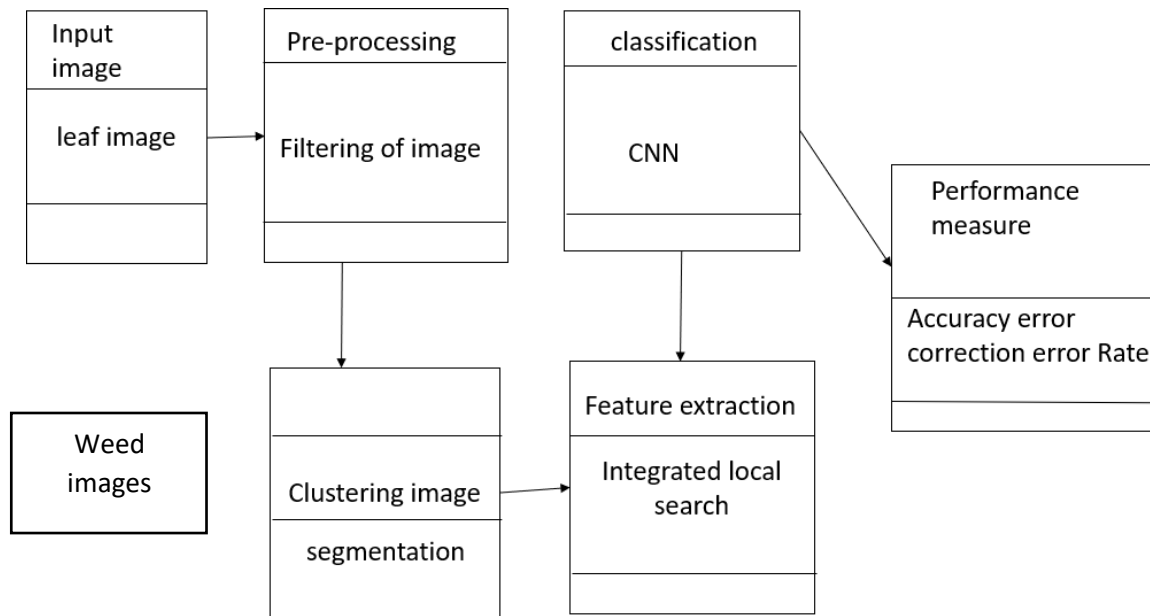


FIG 4.5 Class diagrams

4.3.5 Sequence Diagrams

A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place as shown in Fig

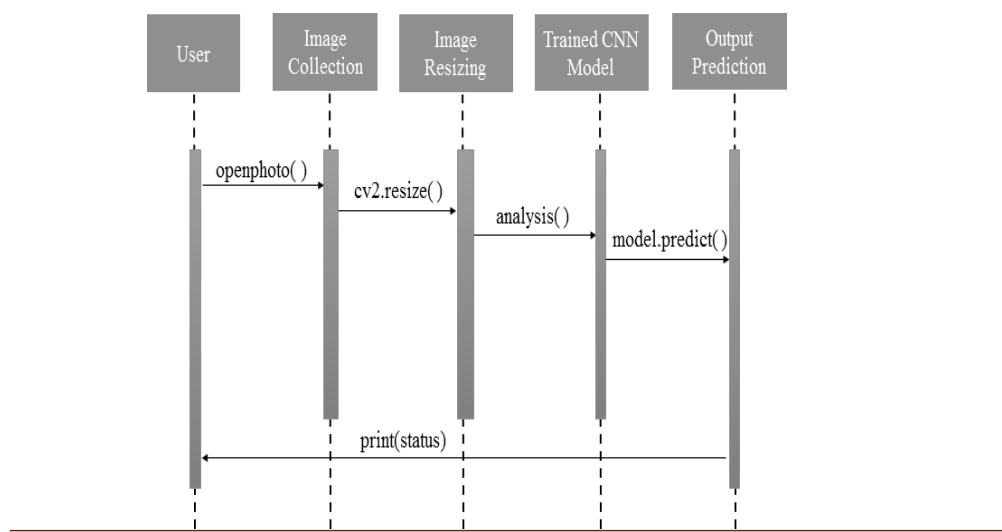


Fig 4.6 sequence diagrams

4.3.6 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

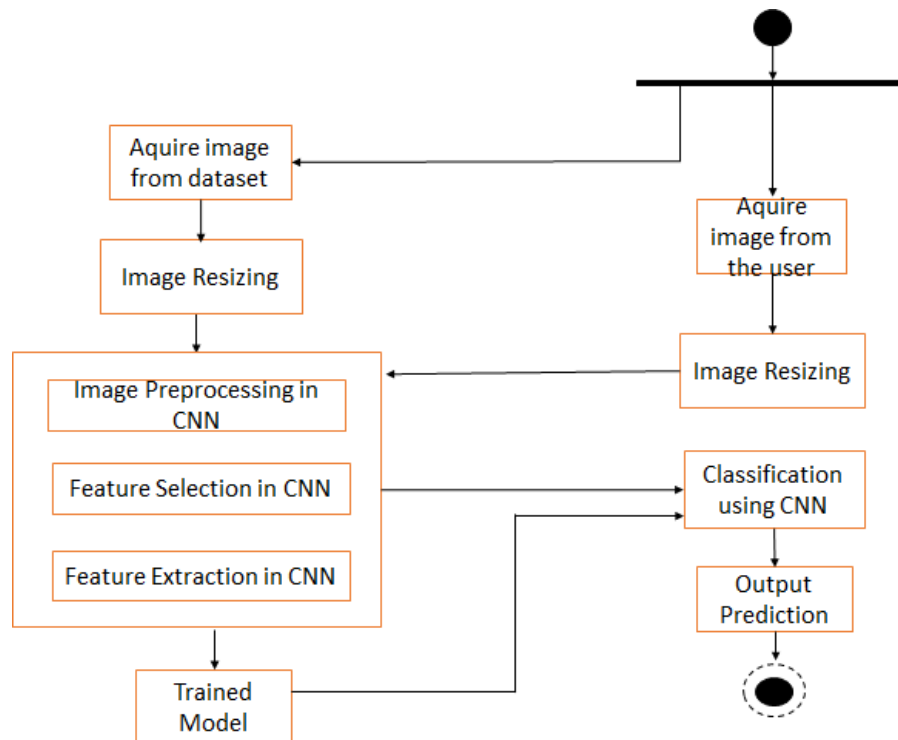


Fig 4.7 activity diagram

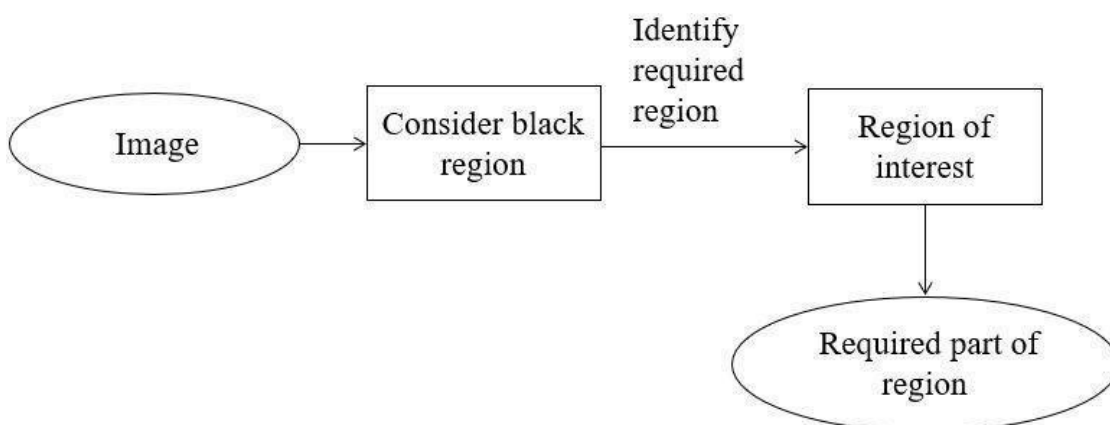


FIG 4.8 Data Flow Diagram for Pre-processing

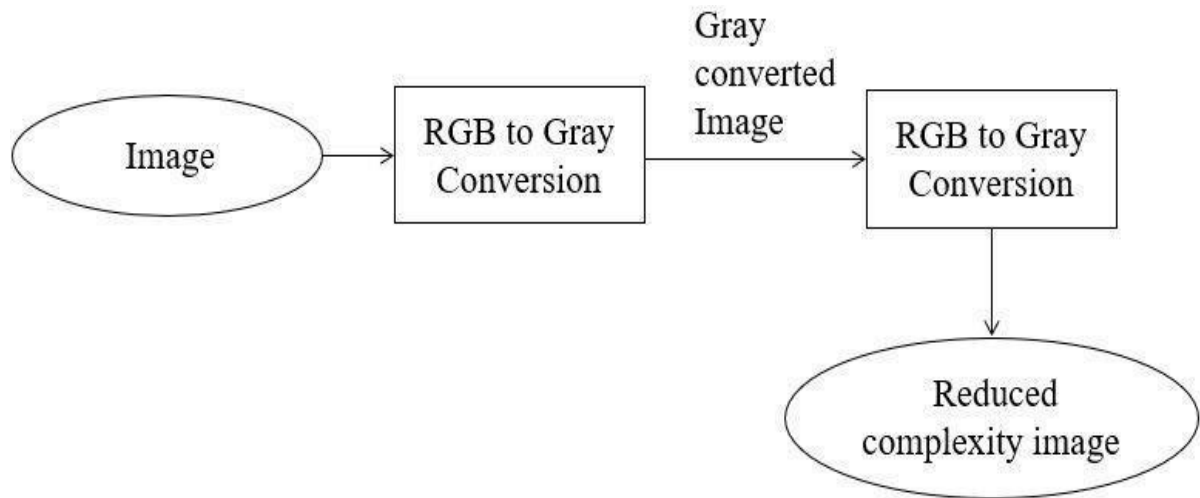


FIG 4.9 Data Flow Diagram for Identification

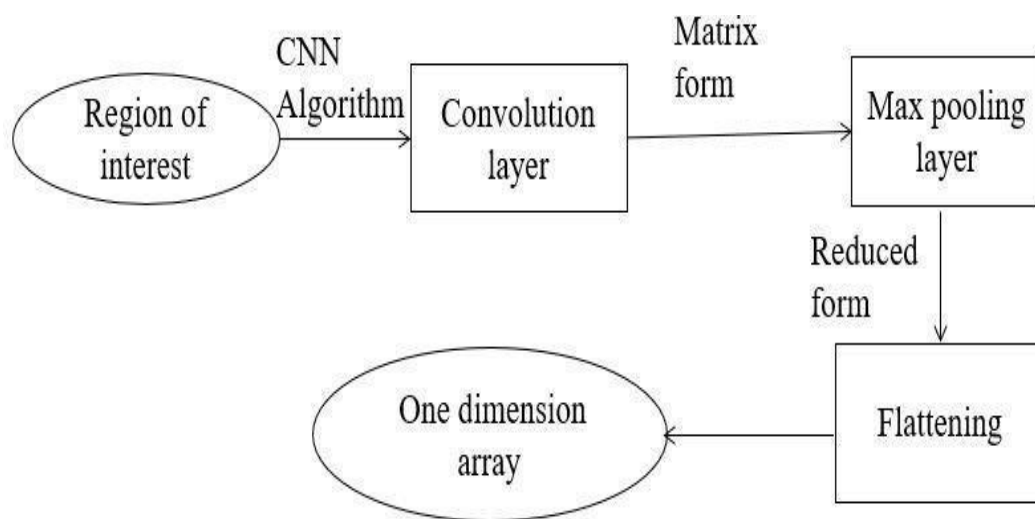


FIG 4.9.1 Data Flow Diagram for Feature Extraction

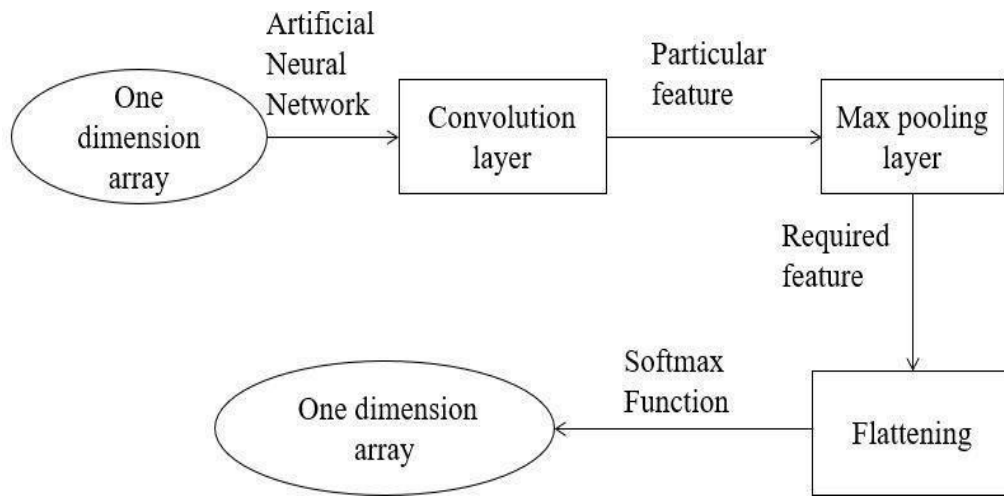


FIG 4.9.2 Data Flow Diagram for Classification and Detection

Specification using Use Case Diagram

The use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.

The use case diagram for tomato leaf disease detection model is as depicted in the figure 3.2. Initially the set of captured images are stored in a temporary file in OpenCV. The obtained RGB image is converted in to gray scale image to reduce complexity. Then the pre-processing techniques are applied on the obtained gray scale image. The feature extraction is carried out then the features are extracted and fed into classifiers which are trained based on the available dataset to detect the disease. The detected disease is displayed along with the intermediate results, various information regarding the detected diseases is also detected.

A formal data collection process is necessary as it ensures that the data gathered are both defined and accurate and that subsequent decisions based on arguments embodied in the findings are valid.

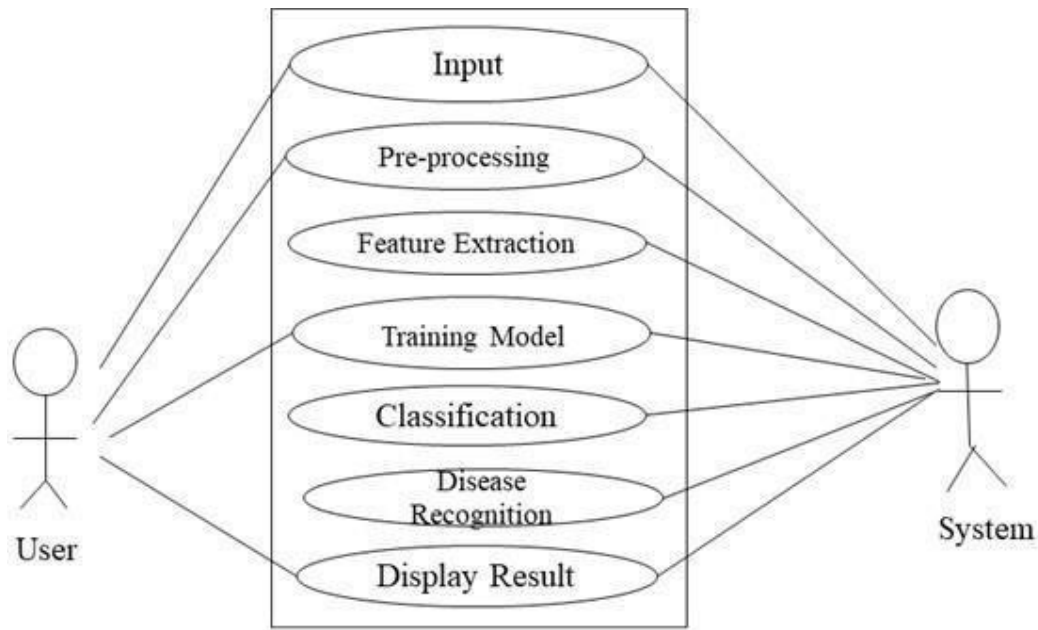


FIG 4.9.3 Use Case Diagram for weed detection

Module Specification

Module Specification is the way to improve the structure design by breaking down the system into modules and solving it as independent task. By doing so the complexity is reduced and the modules can be tested independently.

CHAPTER 5

IMPLEMENTATION

Implementation is the process of converting a new system design into an operational one. It is the key stage in achieving a successful new system. It must therefore be carefully planned and controlled. The implementation of a system is done after the development effort is completed

5.1 Steps for Implementation

Front-End Development Using Python Tkinter:

Modern computer applications are user-friendly. User interaction is not restricted to console-based I/O. They have a more ergonomic graphical user interface (GUI) thanks to high-speed processors and powerful graphics hardware. These applications can receive inputs through mouse clicks and can enable the user to choose from alternatives with the help of radio buttons, dropdown lists, and other GUI elements.

- **Tkinter Programming:**

Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit. Tkinter has several strengths. It's cross-platform, so the same code works on Windows, macOS, and Linux. Visual elements are rendered using native operating system elements, so applications built with Tkinter look like they belong on the platform where they're run.

- **Implementation Issues**

The implementation phase of software development is concerned with translating design specifications into source code. The primary goal of implementation is to write source code and internal documentation so that conformance of the code to its specifications can be easily verified and so that debugging testing and modification are eased. This goal can be achieved by making the source code as clear and straightforward as possible. Simplicity clarity and elegance are the hallmarks of good programs and these characteristics have been implemented in each program module.

- **OpenCV-Python Tool**

OpenCV-Python is a library of Python bindings designed to solve computer vision problems. Visual information is the most important type of information perceived, processed and interpreted by the human brain. Image processing is a method to perform some operations on

an image, in order to extract some useful information from it. An image is nothing more than a two dimensional matrix (3-D in case of colored images) which is defined by the mathematical function $f(x,y)$ where x and y are the two coordinates horizontally and vertically. The value of $f(x,y)$ at any point gives the pixel value at that point of an image, the pixel value describes how bright that pixel is, and/or what color it should be. In image processing we can also perform image acquisition, storage, image enhancement.

- **Flask**

Flask (source code) is a Python web framework built with a small core and easy-to-extend philosophy. Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications. It began as a simple wrapper around Werkzeug and Jinja and has become one of the most popular Python web application frameworks.

Flask offers suggestions, but doesn't enforce any dependencies or project layout. It is up to the developer to choose the tools and libraries they want to use. There are many extensions provided by the community that make adding new functionality easy. Flask is considered more Pythonic than the Django web framework because in common situations the equivalent Flask web application is more explicit. Flask is also easy to get started with as a beginner because there is little boilerplate code for getting a simple app up and running.

Flask was also written several years after Django and therefore learned from the Python community's reactions as the framework evolved. Jökull Sólberg wrote a great piece articulating to this effect in his experience switching between Flask and Django.

- **Google Colab**

Colaboratory, or “Colab” for short, is a product from Google Research. Colab allows anybody to write and execute arbitrary python code through the browser, and is especially well suited to machine learning, data analysis and education. More technically, Colab is a hosted Jupyter notebook service that requires no setup to use, while providing free access to computing resources including GPUs. Colab resources are not guaranteed and not unlimited, and the usage limits sometimes fluctuate. This is necessary for Colab to be able to provide resources for free. Jupyter is the open-source project on which Colab is based. Colab allows you to use and share Jupyter notebooks with others without having to download, install, or run anything. Code is executed in a virtual machine private to your account. Virtual machines are deleted when idle for a while, and have a maximum lifetime enforced by the Colab service.

Resources in Colab are prioritized for users who have recently used less resources, in order to prevent the monopolization of limited resources by a small number of users.

- **Packages**

Packages are the namespaces which consists of multiple packages and module themselves. Each package in Python is a directory which must contain a special file called `_init_.py`. This file can be empty, and it indicates that the directory it contains is a Python package, so it can be imported in the same way that the module can be imported.

As our application program grows larger in size with a lot of modules, one can place the similar modules in one package and different modules in different packages. This makes a program easy to manage and conceptually clear. We can import the modules from packages using the dot (.) operator.

1. csv

In this work, to import or export spread sheets and databases for its use in the Python interpreter, the CSV module, or Comma Separated Values format is used.

These CSV files are used to store a large number of variables or data and the CSV module is a built-in function that allows Python to parse these types of files. The text inside a CSV file is organized in the form of rows, and each row consists of the columns, all separated by commas, indicates the separate cells in CSV file. There is no standard for CSV modules hence, these modules makes use of “dialects” to support parsing using different parameters.

The CSV module includes all the necessary built-in functions, `csv.reader` and `csv.writer` are the most commonly used built-in functions in Python. These functions are given below:

```
csv.reader (csvfile, dialect='excel', **fmtparams)
```

It will return a reader object which will iterate over lines in the given csvfile.

csvfile can be any object which supports the iterator protocol and returns a string each time its `next()` method is called – file objects and list objects are both suitable. If csvfile is a file object, it must be opened with the ‘b’ flag on platforms where that makes a difference. An optional dialect parameter can be given which is used to define a set of parameters specific to a particular CSV dialect. It may be an instance of a subclass of the `Dialect` class or one of the strings returned by the `list_dialects()` function. The other optional `fmtparams` keyword arguments can be given to override individual formatting parameters in the current dialect. Each row read from the csv file is returned as a list of strings. No automatic data type conversion is performed.

```
csv.writer(csvfile, dialect='excel', **fmtparams)
```

It returns a writer object responsible for converting the user's data into delimited strings on the given file-like object. `csvfile` can be any object with a `write()` method. If `csvfile` is a file object, it must be opened with the 'b' flag on platforms where that makes a difference. An optional `dialect` parameter can be given which is used to define a set of dialect. It may be an instance of a subclass of the `Dialect` class or one of the strings returned by the `list_dialects()` function. The other optional `fmtparams` keyword arguments can be given to override individual formatting parameters in the current dialect. Floats are stringified with `repr()` before being written. All other non-string data are stringified with `str()` before being written.

2. TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural network, it is used for both research and production at Google. TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache License 2.0 on November 9, 2015. TensorFlow is Google Brain's second-generation system. Version 1.0.0 was released on February 11, 2017. While the reference implementation runs in single devices, Tensorflow can run on multiple CPUs and GPUs. Tensorflow is available on 64bit Linux, macOS, Windows and mobile computing platforms including Android and ios. Its flexible architecture allows for the easy deployment of computation across a variety of platforms (CPUs, GPUs and TPUs), and from desktops to clusters to servers to mobile and edge devices. Tensorflow computations are expressed as state full dataflow graphs. The name tensorflow derive from the operations that such neural networks perform on multidimensional data arrays, which are referred to as tensors.

3. Keras

Keras is an open source neural- network library written in Python. It is capable of running on top of Tensorflow, Microsoft Cognitive Toolkit, R, Theano or PlaidML. It is designed to enable fast experimentation with deep neural networks. It focuses in being user-friendly, modular, and extensible CUDA. Keras contains numerous implementations of commonly used neural- network building blocks such as layers, objectives, activation

function, optimizers, and a host of tools to make working with image and text data easier to simplify the code necessary for writing deep neural network code. The code is hosted on GitHub, and commonly support forums include the GitHub issues page, and a Slack channel. In addition to Standard neural networks, Keras has support for convolutional and recurrent neural networks. It supports other common utility layers like dropout, batch normalization and pooling. Keras allows users to productize deep models on smart phones, on the web or on the java virtual machine. It also allows use of distributed training of deep learning models on clusters of Graphics processing units (GPU) and tensor processing units (TPU) principally in conjunction with CUDA.

4. NumPy

NumPy is a python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices. NumPy was created in 2005 by Travis Oliphant. It is an open-source project and you can use it freely. NumPy stands for Numerical Python. In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x faster than traditional Python lists. The array object in NumPy is called `ndarray`, it provides a lot of supporting functions that make working with `ndarray` very easy. Arrays are very frequently used in data science, where speed and resources are very important. NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently. This behaviour is called locality of reference in computer science. This is the main reason why NumPy is faster than lists. Also, it is optimized to work with latest CPU architectures.

5. Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Matplotlib produces publication-quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shell, web application servers, and various graphical user interface toolkits. Matplotlib. pyplot is a collection of command style functions that make matplotlib work like MATLAB. Each pyplot function makes some change to a figure: e.g., creates a figure, creates a plotting area in a figure, plots some lines in a plotting area,

decorates the plot with labels, etc. Some people embed matplotlib into graphical user interfaces like wxpython or pygtk to build rich applications. Others use matplotlib in batch scripts to generate postscript images from some numerical simulations, and still others in web application servers to dynamically serve up graphs. Matplotlib is the brainchild of John Hunter (1968-2012), who, along with its many contributors, have put an immeasurable amount of time and effort into producing a piece of software utilized by thousands of scientists worldwide.

The goals of implementation are as follows

- Minimize the memory required.
- Maximize output readability.
- Maximize source text readability.
- Minimize the number of source statements.
- Minimize development time

Module specification:

Module Specification is the way to improve the structure design by breaking down the system into modules and solving it as independent task. By doing so the complexity is reduced and the modules can be tested independently. The number of modules for our model is three, namely preprocessing, identification, feature extraction and detection. So each phase signify the functionalities provided by the proposed system. In the data pre-processing phase noise removal using median filtering is done.

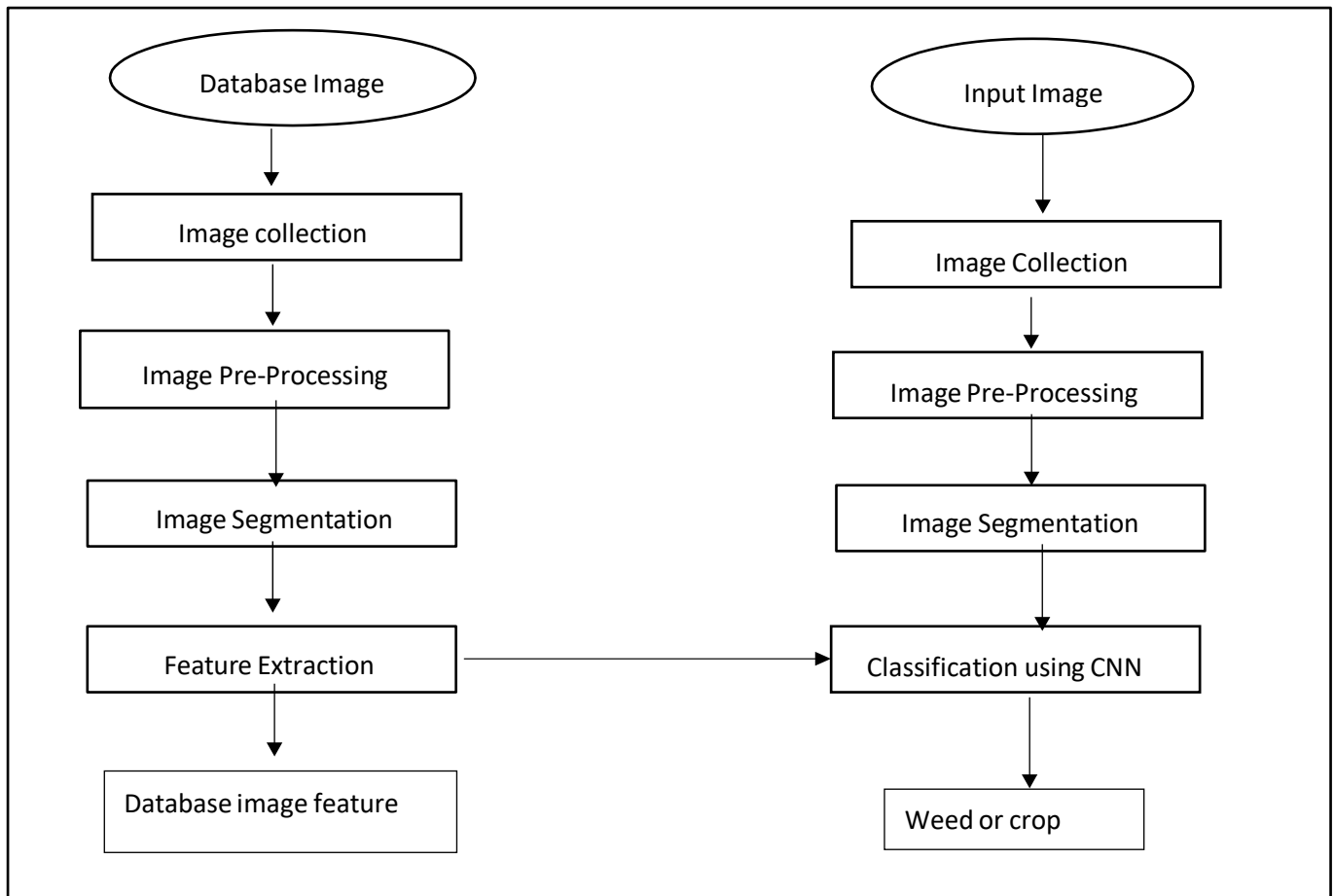


FIG 5.1 Data flow Diagram of Training and Testing Phase

5.2 Modules Description

The System design mainly consists of

1. Image Collection
2. Image Preprocessing
3. Image Segmentation
4. Feature Extraction
5. Training
6. Classification

1. Image Collection

The dataset that we have used in this project is available publicly on the internet

5.2.1 Pre-Processing Module

Name of the Module: Pre-Processing

Actors: User, System

Use Cases: Input Image, Generate RGB Image, RGB to Grey Image, Noise Removal, Thresholding, Disease Recognition, Image Sharpening.

Functionality: The main functionality of this module is to preprocess the data to obtain the features conveniently.

Description: Figure shows the use case diagram of the pre-processing module. In this usecase diagram, there are six use cases and two actors. In the first use case, image is captured and is used as input for this module. In second use case, for the captured image RGB image is generated. In third use case, the RGB image into Gray scale image to reduce complexity. In fourth use case noise removal is done. In fifth use case, thresholding is done. In sixth use case, Image Sharpening is carried out.

Image Preprocessing

The goal of pre-processing is an improvement of image data that reduces unwanted distortions and enhances some image features important for further image processing. Image pre-processing involves three main things a) Grayscale conversion b) Noise removal c) Image enhancement

a) Grayscale conversion: Grayscale image contains only brightness information. Each pixel value in a grayscale image corresponds to an amount or quantity of light. The brightness graduation can be differentiated in grayscale image. Grayscale image measures only light intensity 8 bit image will have brightness variation from 0 to 255 where '0' represents black and '255' represent white. In grayscale conversion color image is converted into grayscale image shows. Grayscale images are easier and faster to process than colored images. All image processing technique are applied on grayscale image.

b) Noise Removal: The objective of noise removal is to detect and remove unwanted noise from digital image. The difficulty is in deciding which features of an image are real and which are caused by noise. Noise is random variations in pixel values.

We are using median filter to remove unwanted noise. Median filter is nonlinear filter, it leaves edges invariant. Median filter is implemented by sliding window of odd length. Each sample value is sorted by magnitude, the centermost value is median of sample within the window, is a filter output.

5.2.2 Image Enhancement: The objective of image enhancement is to process an image to increase visibility of feature of interest. Here contrast enhancement is used to get better

quality result.

5.2.3 Image Segmentation Image segmentation are of many types such as clustering, threshold, neural network based and edge based. In this implementation we are using the clustering algorithm called mean shift clustering for image segmentation. This algorithm uses the sliding window method for converging to the Centre of maximum dense area. This algorithm makes use of many sliding windows to converge the maximum dense region. Mean shift clustering Algorithm This algorithm is mainly used for detecting highly dense region.

5.2.3 Feature Extraction:

There are many features of an image mainly color, texture, and shape. Here we are considering three features that are color histogram, Texture which resembles color, shape, and texture.

5. Training Training dataset was created from images of known Cancer stages. Classifiers are trained on the created trainingdataset. A testing dataset is placed in a temporary folder. Predicted results from the test case, Plots classifiers graphs and add feature-sets to test case file, to make image processing models more accurate

5.2.4 Classification

The binary classifier which makes use of the hyper-plane which is also called as the decision boundary between two of the classes is called as Convolution Neural Network. Some of the problems are pattern recognition like texture classification makes use of CNN. Mapping of nonlinear input data to the linear data provides good classification in high dimensional space in CNN. The marginal distance is maximized between different classes by CNN. Different Kernels are used to divide the classes. CNN is basically a binary classifier that determines hyper plane in dividing two classes. The boundary is maximized between the hyperplane and two classes. The samples that are nearest to the margin will be selected in determining the hyperplane is called support vectors.

CNN Algorithm Explanation

The first neural network named LeNet5 had a very less validation accuracy of 42% since then we have come a long way in this field. Nowadays almost every giant technology firms rely on CNN for more efficient performance. The idea to detect diseases in mulberry leaf incorporates the use of CNN before we dive into the “functionality and working of CNN” concept, we must have a basic idea on how the human brain recognizes an object in spite of its varying attributes from one another. Our brain has a complex layer of neurons, each layer holds some information about the object and all the features of the object are extracted by the

neurons and stored in our memory, next time when we see the same object the brain matches the stored features to recognize the object, but one can easily mistake it as a simple “IF-THEN” function, yes it is to some extent but it has an extra feature that gives it an edge over other algorithms that is Self-Learning, although it cannot match a human brain but still it can give it a tough competition. Image is processed using the Basic CNN to detect the diseases in leaves.

The data training in our CNN model has to satisfy following constraints: 1) No missing values in dataset. 2) The dataset must distinctly be divided into training and testing sets, either the training or the testing set shouldn't contain any irrelevant data out of our model domain in case of an image dataset all the images must be of the same size, one uneven distribution of image size in our dataset can decrease the efficiency of our neural network. 3) The images should be converted into black and white format before feeding it into the convolution layer because reading images in RGB would involve a 3-D NumPy matrix which will reduce the execution time of our model by a considerable amount. Any kind of corrupted or blurred images should also be trimmed from the database before feeding it into the neural network. This is all about data pre-processing rules, let us dive right into the working of the convolution neural network.

- **Convolution Neural Network layers**

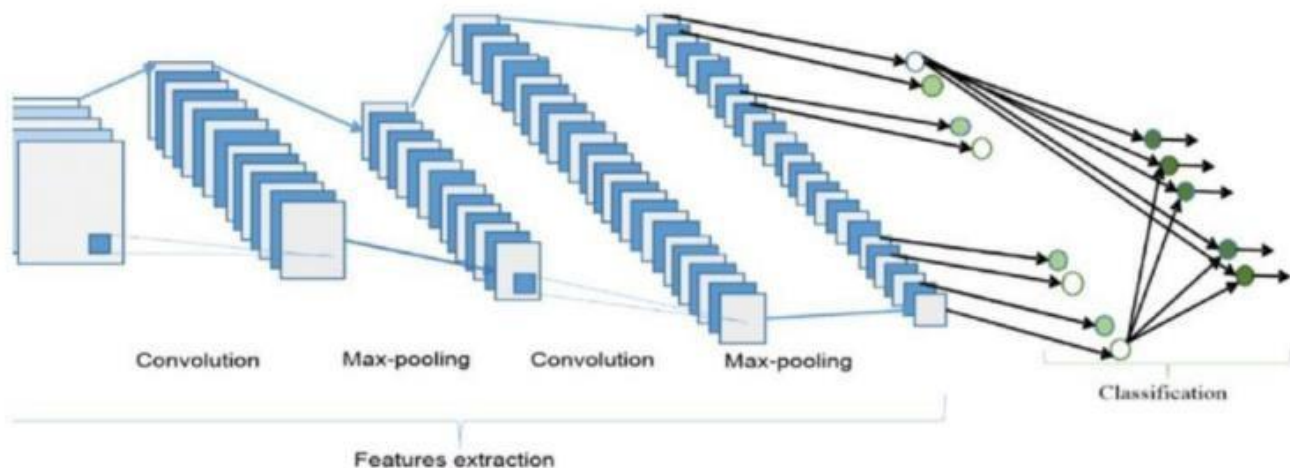


FIG 5.2 Convolutional Neural Network Layers

Convolution layer

This layer involves scanning the whole image for patterns and formulating it in the form of a 3×3 matrix. This convolved feature matrix of the image is known as Kernel. Each value in the kernel is known as a weight vector.



A. Pooling layer

After the convolution comes to the pooling here the image matrix is broken down into the sets of 4 rectangular segments which are non-overlapping. There are two types of pooling, Max pooling and average pooling. Max pooling gives the maximum value in the relative matrix region which is taken. Average pooling gives the average value in relative matrix region. The main advantage of the pooling layer is that it increases computer performance and decreases over-fitting chances.

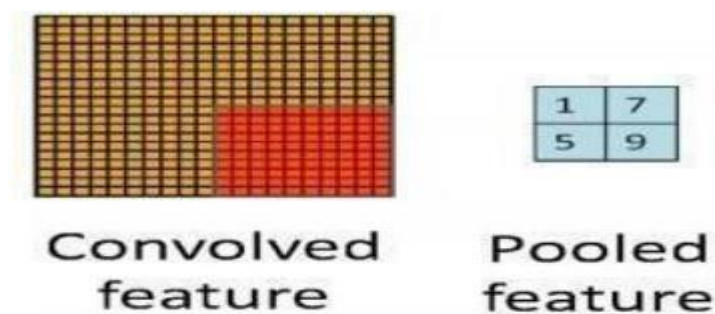


FIG 5.3 Structure of CNN in matrix formatC. Activation layer

Implementation It is the part of the Convolutional Neural Networks where the values are Normalized that is, they are fitted in a certain range. The used convolutional function is ReLU which allows only the positive values and then rejects the negative values. It is the function of low computational cost.

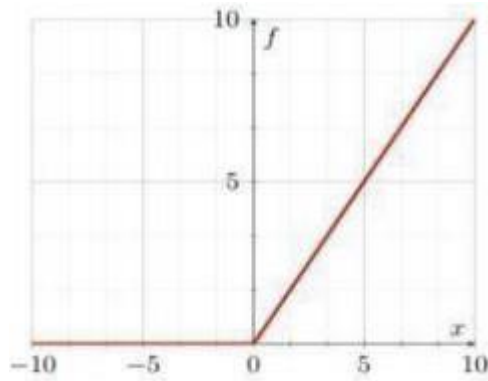


FIG 5.4 Graph of positive ReLU Func

5.3 CODE

```
import math
import argparse
import os
import sys
from pathlib import Path
import time
import cv2
import torch
import torch.backends.cudnn as cudnn

from models.common
import DetectMultiBackend
from utils.datasets
import IMG_FORMATS, VID_FORMATS, LoadImages, LoadStreams
from utils.general
import (check_file, check_img_size, check_imshow, increment_path, non_max_suppression,
scale_coords, strip_optimizer, xyxy2xywh)
from utils.plots import Annotator, colors
from utils.torch_utils import select_device
@torch.no_grad()
def run(weights='weed.pt', # model.pt path(s)
```

```
source='0', # file/dir/URL/glob, 0 for webcam
data='data.yaml', # dataset.yaml path
imgsz=(640, 640), # inference size (height, width)
conf_thres=0.75, # confidence threshold
iou_thres=0.45, # NMS IOU threshold
max_det=1, # maximum detections per image
device='', # cuda device, i.e. 0 or 0,1,2,3 or cpu
view_img=False, # show results
save_conf=False, # save confidences in --save-txt labels
classes=None, # filter by class: --class 0, or --class 0 2 3
agnostic_nms=False, # class-agnostic NMS
augment=False, # augmented inference
visualize=False, # visualize features
line_thickness=3, # bounding box thickness (pixels)
hide_labels=False, # hide labels
hide_conf=False, # hide confidences
half=False, # use FP16 half-precision inference
dnn=False, # use OpenCV DNN for ONNX inference
):
```

```
source = str(source)
is_file = Path(source).suffix[1:] in (IMG_FORMATS + VID_FORMATS)
is_url = source.lower().startswith(('rtsp://', 'rtmp://', 'http://', 'https://'))
webcam = source.isnumeric() or source.endswith('.txt') or (is_url and not is_file)
if is_url and is_file:
    source = check_file(source) # download

# Load model
device = select_device(device)
model = DetectMultiBackend(weights, device=device, dnn=dnn, data=data)
stride, names, pt, jit, onnx, engine = model.stride, model.names, model.pt, model.jit,
model.onnx, model.engine
```

```
imgsz = check_img_size(imgsz, s=stride) # check image size

# Half
half &= (pt or jit or onnx or engine) and device.type != 'cpu' # FP16 supported on limited
backends with CUDA

if pt or jit:
    model.model.half() if half else model.model.float()

# Dataloader
if webcam:
    view_img = check_imshow()
    cudnn.benchmark = True # set True to speed up constant image size inference
    dataset = LoadStreams(source, img_size=imgsz, stride=stride, auto=pt)
    bs = len(dataset) # batch_size
else:
    dataset = LoadImages(source, img_size=imgsz, stride=stride, auto=pt)
    bs = 1 # batch_size
vid_path, vid_writer = [None] * bs, [None] * bs

# Run inference
for path, im, im0s, vid_cap, s in dataset:
    im = torch.from_numpy(im).to(device)
    im = im.half() if half else im.float() # uint8 to fp16/32
    im /= 255 # 0 - 255 to 0.0 - 1.0

    if len(im.shape) == 3:
        im = im[None] # expand for batch dim

    # Inference
    visualize = increment_path(save_dir / Path(path).stem, mkdir=True) if visualize else
False
    pred = model(im, augment=augment, visualize=visualize)
    # NMS
```

```
pred = non_max_suppression(pred, conf_thres, iou_thres, classes, agnostic_nms,
max_det=max_det)

# Process predictions
for i, det in enumerate(pred): # per image
    if webcam:
        im0, frame = im0s[i].copy(), dataset.count
        #s += f'{i}: '
    else:
        im0, frame = im0s.copy(), getattr(dataset, 'frame', 0)

    gn = torch.tensor(im0.shape)[[1, 0, 1, 0]] # normalization gain whwh
    #imc = im0.copy() if save_crop else im0 # for save_crop
    annotator = Annotator(im0, line_width=line_thickness, example=str(names))

    if len(det):
        # Rescale boxes from img_size to im0 size
        det[:, :4] = scale_coords(im.shape[2:], det[:, :4], im0.shape).round()

        height, width, _ = im0.shape
        # Print results
        for c in det[:, -1].unique():
            n = (det[:, -1] == c).sum() # detections per class

        # Write results
        for *xyxy, conf, cls in reversed(det):
            xywh = (xyxy2xywh(torch.tensor(xyxy).view(1, 4)) / gn).view(-1).tolist() #
normalized xywh
            line = (cls, *xywh, conf) if save_conf else (cls, *xywh) # label format

            c = int(cls) # integer class

            {conf:.2f}')
```

```
if names[c] == 'weed':
    label = None if hide_labels else (names[c] if hide_conf else f'{names[c]}')
    annotator.box_label(xyxy, label, color=colors(c, True))
```

```
# Stream results
im0 = annotator.result() #if view_img:
cv2.imshow('Weed Detection', im0) cv2.imwrite('result.jpg', im0) cv2.waitKey(1)
```

```
if __name__ == "__main__": run()
```

```
from flask import Flask, request, render_template
import os
```

```
app = Flask(__name__)
```

```
@app.route('/')
def index():
```

```
    return render_template('index.html')
```

```
@app.route('/analyse', methods=['GET', 'POST'])
def analyse():
```

```
    if request.method == 'POST':
```

```
        File = request.form['file']
```

```
        from detect import Start
```

```
        Start('static/'+File)
```

```
        return render_template('index.html', image1='static/'+File, image2='static/result.jpg')
```

```
    return render_template('index.html')
```

```
@app.route('/Live')
def Live():
```

```
    from detect_live import Start
```

```
    Start(0)
```

```
    return render_template('index.html')
```



```
if __name__ == "__main__":
    app.run(debug=True)

import argparse
import os
import platform
import sys
from pathlib import Path

import torch

FILE = Path(__file__).resolve()
ROOT = FILE.parents[0] # YOLOv5 root directory
if str(ROOT) not in sys.path:
    sys.path.append(str(ROOT)) # add ROOT to PATH
ROOT = Path(os.path.relpath(ROOT, Path.cwd())) # relative

from models.common import DetectMultiBackend
from utils.dataloaders import IMG_FORMATS, VID_FORMATS, LoadImages, LoadScreenshots,
LoadStreams
from utils.general import (LOGGER, Profile, check_file, check_img_size, check_imshow,
check_requirements, colorstr, cv2,
                           increment_path, non_max_suppression, print_args, scale_boxes, strip_optimizer,
                           xyxy2xywh)
from utils.plots import Annotator, colors, save_one_box
from utils.torch_utils import select_device, smart_inference_mode

@smart_inference_mode()
def run(
    weights=ROOT / 'yolov5s.pt', # model path or triton URL
    source=ROOT / 'data/images', # file/dir/URL/glob/screen/0(webcam)
    data=ROOT / 'data/coco128.yaml', # dataset.yaml path
    imgsz=(640, 640), # inference size (height, width)
    conf_thres=0.25, # confidence threshold
```

```
iou_thres=0.45, # NMS IOU threshold
max_det=1000, # maximum detections per image
device="", # cuda device, i.e. 0 or 0,1,2,3 or cpu
view_img=False, # show results
save_txt=False, # save results to *.txt
save_conf=False, # save confidences in --save-txt labels
save_crop=False, # save cropped prediction boxes
nosave=False, # do not save images/videos
classes=None, # filter by class: --class 0, or --class 0 2 3
agnostic_nms=False, # class-agnostic NMS
augment=False, # augmented inference
visualize=False, # visualize features
update=False, # update all models
project=ROOT / 'runs/detect', # save results to project/name
name='exp', # save results to project/name
exist_ok=False, # existing project/name ok, do not increment
line_thickness=3, # bounding box thickness (pixels)
hide_labels=False, # hide labels
hide_conf=False, # hide confidences
half=False, # use FP16 half-precision inference
dnn=False, # use OpenCV DNN for ONNX inference
vid_stride=1, # video frame-rate stride
):
    source = str(source)
    save_img = not nosave and not source.endswith('.txt') # save inference images
    is_file = Path(source).suffix[1:] in (IMG_FORMATS + VID_FORMATS)
    is_url = source.lower().startswith(('rtsp://', 'rtmp://', 'http://', 'https://'))
    webcam = source.isnumeric() or source.endswith('.streams') or (is_url and not is_file)
    screenshot = source.lower().startswith('screen')
    if is_url and is_file:
        source = check_file(source) # download

    # Directories
    save_dir = increment_path(Path(project) / name, exist_ok=exist_ok) # increment run
    (save_dir / 'labels' if save_txt else save_dir).mkdir(parents=True, exist_ok=True) # make dir
```

```
# Load model
device = select_device(device)
model = DetectMultiBackend(weights, device=device, dnn=dnn, data=data, fp16=half)
stride, names, pt = model.stride, model.names, model.pt
imgsz = check_img_size(imgsz, s=stride) # check image size

# Dataloader
bs = 1 # batch_size
if webcam:
    view_img = check_imshow(warn=True)
    dataset = LoadStreams(source, img_size=imgsz, stride=stride, auto=pt, vid_stride=vid_stride)
    bs = len(dataset)
elif screenshot:
    dataset = LoadScreenshots(source, img_size=imgsz, stride=stride, auto=pt)
else:
    dataset = LoadImages(source, img_size=imgsz, stride=stride, auto=pt, vid_stride=vid_stride)
vid_path, vid_writer = [None] * bs, [None] * bs

# Run inference
model.warmup(imgsz=(1 if pt or model.triton else bs, 3, *imgsz)) # warmup
seen, windows, dt = 0, [], (Profile(), Profile(), Profile())
for path, im, im0s, vid_cap, s in dataset:
    with dt[0]:
        im = torch.from_numpy(im).to(model.device)
        im = im.half() if model.fp16 else im.float() # uint8 to fp16/32
        im /= 255 # 0 - 255 to 0.0 - 1.0
        if len(im.shape) == 3:
            im = im[None] # expand for batch dim

# Inference
with dt[1]:
    visualize = increment_path(save_dir / Path(path).stem, mkdir=True) if visualize else False
    pred = model(im, augment=augment, visualize=visualize)
```

```
# NMS
with dt[2]:
    pred = non_max_suppression(pred, conf_thres, iou_thres, classes, agnostic_nms,
max_det=max_det)

# Second-stage classifier (optional)
# pred = utils.general.apply_classifier(pred, classifier_model, im, im0s)

# Process predictions
for i, det in enumerate(pred): # per image
    seen += 1
    if webcam: # batch_size >= 1
        p, im0, frame = path[i], im0s[i].copy(), dataset.count
        s += f'{i}: '
    else:
        p, im0, frame = path, im0s.copy(), getattr(dataset, 'frame', 0)

    p = Path(p) # to Path
    save_path = str(save_dir / p.name) # im.jpg
    txt_path = str(save_dir / 'labels' / p.stem) + (' if dataset.mode == 'image' else f'_{frame}') #
im.txt
    s += '%gx%g ' % im.shape[2:] # print string
    gn = torch.tensor(im0.shape)[[1, 0, 1, 0]] # normalization gain whwh
    imc = im0.copy() if save_crop else im0 # for save_crop
    annotator = Annotator(im0, line_width=line_thickness, example=str(names))
    if len(det):
        # Rescale boxes from img_size to im0 size
        det[:, :4] = scale_boxes(im.shape[2:], det[:, :4], im0.shape).round()

        # Print results
        for c in det[:, 5].unique():
            n = (det[:, 5] == c).sum() # detections per class
            s += f'{n} {names[int(c)]}' * (n > 1), " # add to string

    # Write results
```

```
for *xyxy, conf, cls in reversed(det):
    if save_txt: # Write to file
        xywh = (xyxy2xywh(torch.tensor(xyxy).view(1, 4)) / gn).view(-1).tolist() # normalized
xywh

    line = (cls, *xywh, conf) if save_conf else (cls, *xywh) # label format
    with open(f'{txt_path}.txt', 'a') as f:
        f.write((' %g ' * len(line)).rstrip() % line + '\n')

    if save_img or save_crop or view_img: # Add bbox to image
        c = int(cls) # integer class
        label = None if hide_labels else (names[c] if hide_conf else f'{names[c]} {conf:.2f}')
        annotator.box_label(xyxy, label, color=colors(c, True))
        # kavan.sendMessage(chatid,str("Detected : "+str(names[c])))
    if save_crop:
        save_one_box(xyxy, imc, file=save_dir / 'crops' / names[c] / f'{p.stem}.jpg', BGR=True)

# Stream results
im0 = annotator.result()
if view_img:
    if platform.system() == 'Linux' and p not in windows:
        windows.append(p)
        cv2.namedWindow(str(p), cv2.WINDOW_NORMAL | cv2.WINDOW_KEEPRATIO) #
allow window resize (Linux)
        cv2.resizeWindow(str(p), im0.shape[1], im0.shape[0])
        cv2.imshow(str(p), im0)
        cv2.waitKey(1) # 1 millisecond

# Save results (image with detections)
if save_img:
    if dataset.mode == 'image':
        cv2.imwrite(save_path, im0)
        cv2.imwrite("static/result.jpg",im0)
    else: # 'video' or 'stream'
        if vid_path[i] != save_path: # new video
            vid_path[i] = save_path
```

```
if isinstance(vid_writer[i], cv2.VideoWriter):
    vid_writer[i].release() # release previous video writer
if vid_cap: # video
    fps = vid_cap.get(cv2.CAP_PROP_FPS)
    w = int(vid_cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    h = int(vid_cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
else: # stream
    fps, w, h = 30, im0.shape[1], im0.shape[0]
save_path = str(Path(save_path).with_suffix('.mp4')) # force *.mp4 suffix on results
videos
vid_writer[i] = cv2.VideoWriter(save_path, cv2.VideoWriter_fourcc(*'mp4v'), fps, (w,
h))
vid_writer[i].write(im0)

# Print time (inference-only)
LOGGER.info(f'{s}{" if len(det) else '(no detections), '}{dt[1].dt * 1E3:.1f}ms")

# Print results
t = tuple(x.t / seen * 1E3 for x in dt) # speeds per image
LOGGER.info(f'Speed: %.1fms pre-process, %.1fms inference, %.1fms NMS per image at shape {(1,
3, *imgsz)}' % t)
if save_txt or save_img:
    s = f"\n{len(list(save_dir.glob('labels/*.txt')))} labels saved to {save_dir / 'labels'}" if save_txt else ""
    LOGGER.info(f'Results saved to {colorstr('bold', save_dir)}{s}')
if update:
    strip_optimizer(weights[0]) # update model (to fix SourceChangeWarning)

def parse_opt(FILE):
    parser = argparse.ArgumentParser()
    parser.add_argument('--weights', nargs='+', type=str, default=ROOT / 'best.pt', help='model path or
triton URL')
    parser.add_argument('--source', type=str, default=FILE, help='file/dir/URL/glob/screen/0(webcam)')
    parser.add_argument('--data', type=str, default=ROOT / 'data/coco128.yaml', help='(optional)')
```

```
dataset.yaml path')
```

```
    parser.add_argument('--imgsz', '--img', '--img-size', nargs='+', type=int, default=[640],
help='inference size h,w')
    parser.add_argument('--conf-thres', type=float, default=0.40, help='confidence threshold')
    parser.add_argument('--iou-thres', type=float, default=0.45, help='NMS IoU threshold')
    parser.add_argument('--max-det', type=int, default=1000, help='maximum detections per image')
    parser.add_argument('--device', default="", help='cuda device, i.e. 0 or 0,1,2,3 or cpu')
    parser.add_argument('--view-img', action='store_true', help='show results')
    parser.add_argument('--save-txt', action='store_true', help='save results to *.txt')
    parser.add_argument('--save-conf', action='store_true', help='save confidences in --save-txt labels')
    parser.add_argument('--save-crop', action='store_true', help='save cropped prediction boxes')
    parser.add_argument('--nosave', action='store_true', help='do not save images/videos')
    parser.add_argument('--classes', nargs='+', type=int, help='filter by class: --classes 0, or --classes 0 2
3')
    parser.add_argument('--agnostic-nms', action='store_true', help='class-agnostic NMS')
    parser.add_argument('--augment', action='store_true', help='augmented inference')
    parser.add_argument('--visualize', action='store_true', help='visualize features')
    parser.add_argument('--update', action='store_true', help='update all models')
    parser.add_argument('--project', default=ROOT / 'runs/detect', help='save results to project/name')
    parser.add_argument('--name', default='exp', help='save results to project/name')
    parser.add_argument('--exist-ok', action='store_true', help='existing project/name ok, do not
increment')
    parser.add_argument('--line-thickness', default=3, type=int, help='bounding box thickness (pixels)')
    parser.add_argument('--hide-labels', default=False, action='store_true', help='hide labels')
    parser.add_argument('--hide-conf', default=False, action='store_true', help='hide confidences')
    parser.add_argument('--half', action='store_true', help='use FP16 half-precision inference')
    parser.add_argument('--dnn', action='store_true', help='use OpenCV DNN for ONNX inference')
    parser.add_argument('--vid-stride', type=int, default=1, help='video frame-rate stride')
    opt = parser.parse_args()
    opt.imgsz *= 2 if len(opt.imgsz) == 1 else 1 # expand
    print_args(vars(opt))
    return opt
```

```
def main(opt):
```

```
check_requirements(exclude=('tensorboard', 'thop'))  
run(**vars(opt))
```

```
def Start (FILE):  
    opt = parse_opt(FILE)  
    main(opt)
```


CHAPTER 6

SYSTEM TESTING

Testing is the process of evaluating a system or its component(s) with the intent to find whether it satisfies the specified requirements or not. Testing is executing a system to identify any gaps, errors, or missing requirements in contrary to the actual requirements. System testing of a software or hardware is a testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of black-box testing, and such, should require no knowledge of the inner design of the code or logic. As a rule, system testing takes, as its input, of all the 'integrated' software components that have passed integration testing and the software system itself integrated with any applicable software systems.

The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together. System testing is a more limited type of testing. It seeks to detect defects both within the inter-assemblages and within the system. System testing is performed on the entire system in the context of a Functional Requirement Specification (FRS) and / or a System Requirement Specification (SRS). System testing tests not only the design, but also the behaviour and even the believed expectation of the customer. It is also intended to test up to and beyond the bounds defined in the software / hardware requirement specification. Before applying methods to design effective test cases, a software engineer must understand the basic principle that guides software testing. All the tests should be traceable to customer requirements.

Types of testing

Software testing methods are traditionally divided into two: white-box and black-box testing. These two approaches are used to describe the point of view that a test engineer takes when designing test cases.

6.2 White-box testing

(also known as clear box testing, glass box testing, transparent box testing and structural testing, by seeing the source code) tests internal structures or workings of a program, as opposed to the functionality exposed to the end-user. In white-box testing an internal perspective of the system, as well as programming skills, are used to design test cases. The tester chooses inputs to exercise paths through the code and determine the appropriate

outputs. While white-box testing can be applied at the unit, integration, and system levels of the software testing process, it is usually done at the unit level. It can test paths within a unit, paths between units during integration, and between sub systems during a system-level test. Though this method of test design can uncover many errors or problems, it might not detect unimplemented parts of the specification or missing requirements.

White Box Testing

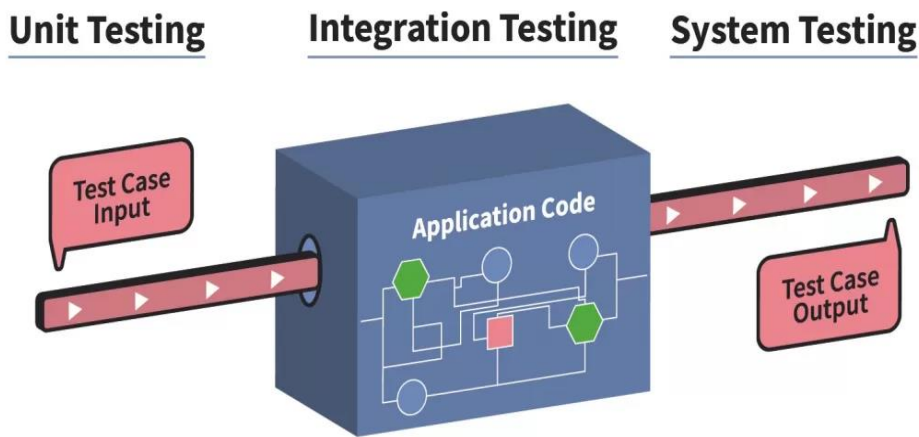


FIG 6.1 White box testing

6.3. Black box testing: The technique of testing without having any knowledge of the interior workings of the application is called black-box testing. The tester is oblivious to the system architecture and does not have access to the source code. Typically, while performing a black-box test, a tester will interact with the system's user interface by providing inputs and examining outputs without knowing how and where the inputs are worked upon. This project has been tested under different circumstances, which includes different types such as Unit testing, Integration testing and System testing that are described below.

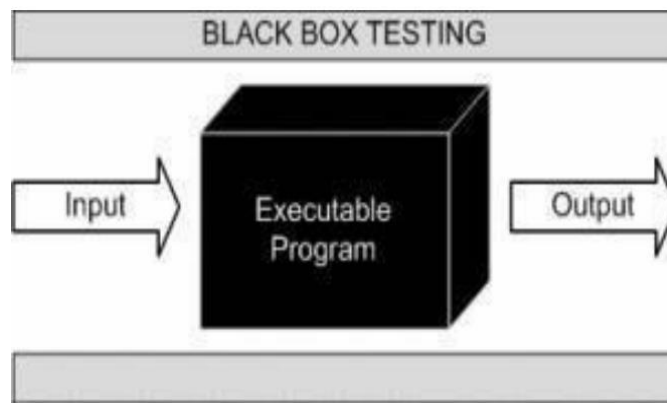


FIG 6.2 Black box testing

Levels of Testing:

There are different levels during the process of testing. Levels of testing include different methodologies that can be used while conducting software testing. The main levels of software testing are:

Functional Testing: This is a type of black-box testing that is based on the specifications of the software that is to be tested. The application is tested by providing input and then the results are examined that need to conform to the functionality it was intended for. Functional testing of software is conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. There are five steps that are involved while testing an application for functionality.

- The determination of the functionality that the intended application is meant to perform.
- The creation of test data based on the specifications of the application.
- The output based on the test data and the specifications of the application.
- The writing of test scenarios and the execution of testcases.
- The comparison of actual and expected results based on the executed testcases.

Non-functional Testing: This section is based upon testing an application from its non-functional attributes. Nonfunctional testing involves testing software from the requirements which are non-functional in nature but important such as performance, security, user interface, etc.

6.4 Unit Testing

Unit testing is a method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures and operating procedures are tested to determine if they are fit for use. Intuitively, one can view a unit as the smallest testable part of an application. During the development process itself all the syntax errors etc. got rooted out. For this developed test case that result in executing every instruction in the program or module i.e. every path through program was tested. Test cases are data chosen at random to check every possible branch after all the loops.

- **Unit Testing Test Case 1**

Sl # Test Case : -	UTC-1
Name of Test: -	Uploading image
Items being tested: -	Tested for uploading different images
Sample Input: -	Upload Sample image
Expected output: -	Image should upload properly
Actual output: -	upload successful
Remarks: -	Pass.

- **Unit Testing Test Case 2**

S1 # Test Case:	UTC-2
Name of Test	Detecting weed images
Items being tested	Test for different weed images
Sample Input	Tested for different images of weed
Expected output	Weed should be displayed
Actual output	Should Display weed or crop
Remarks	Predicted result

6.5 Integration Testing:

Integration testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in Integration Testing. Integration testing is defined as the testing of combined parts of an application to determine if they function correctly. It occurs after unit testing and before validation testing. Integration testing can be done in two ways: Bottom-up integration testing and Top-down integration testing.

1.**Bottom-up Integration:** This testing begins with unit testing, followed by tests of progressively higher-level combinations of units called modules or builds.

2.**Top-down Integration:** In this testing, the highest-level modules are tested first and progressively, lower-level modules are tested thereafter. In a comprehensive software development environment, bottom-up testing is usually done first, followed by top-down testing. The process concludes with multiple tests of the complete application, preferably in scenarios designed to mimic actual situations.

Functional Testing Test Case 1

S1 # Test Case:	ITC
Name of Test	Working of choose file option
Items being tested	User convenience to access images stored
Sample Input	Click and select the image
Expected output	Should open selected Image
Actual output	Selected open selected image
Remarks	Pass.

Functional Testing Test Case 2

S1 # Test Case:	ITC-2
Name of Test	Working of Segmentation and Displaying weed
Items being tested	Selecting different images and verifying weed
Sample Input	Click and select the image
Expected output	Should show weed and predict weed or crop
Actual output	Image Segmented, weed detected should be displayed
Remarks	Pass.

6.6 System testing:

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of black-box testing, and as such, should require no knowledge of the inner design of the code or logic. System testing is important because of the following reasons:

System testing is the first step in the Software Development Life Cycle, where the application is tested.

- The application is tested thoroughly to verify that it meets the functional and technical specifications.
- The application is tested in an environment that is very close to the production environment where the application will be deployed.
- System testing enables us to test, verify, and validate both the business requirements as well as the application architecture.

System Testing is shown in the below tables

S1 # Test Case:	STC-1
Name of Test	System testing in various versions of OS
Items being tested	OS compatibility.
Sample Input	Execute the program in windows XP/Windows-10
Expected output	Performance is better in windows-10
Actual output	Same as expected output,performance is better in windows-10
Remarks	Pass.

CHAPTER 7

EXPECTED OUTCOMES/RESULTS

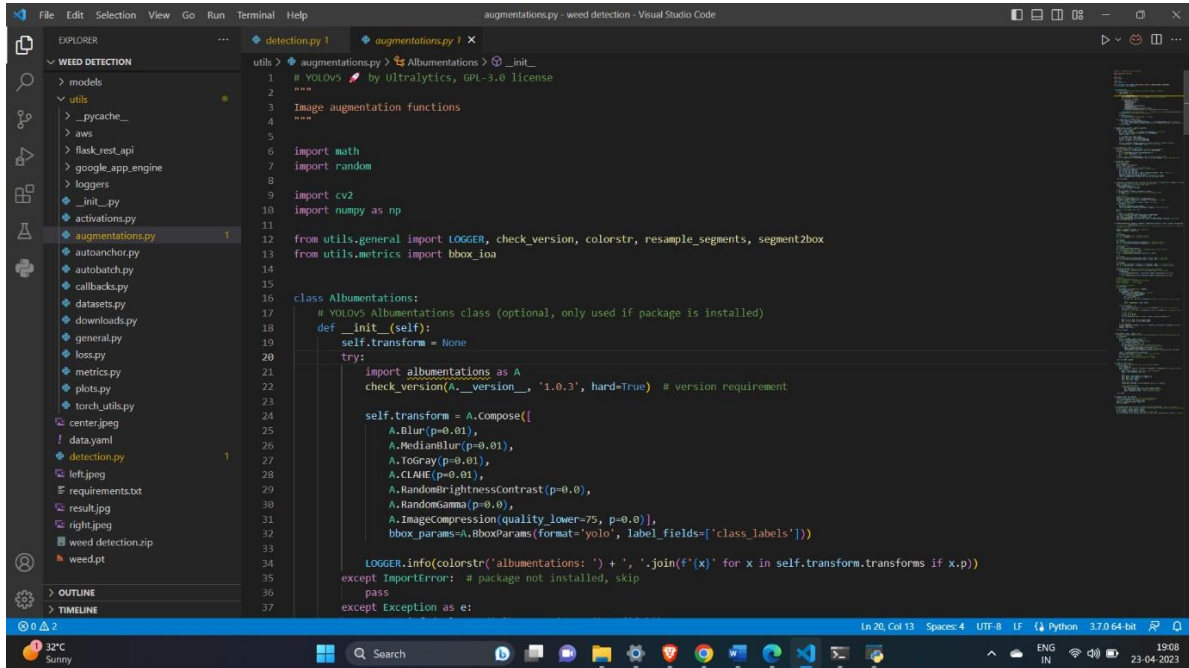


FIG 7.1 IMPLEMENTATION OF SOURCE CODE

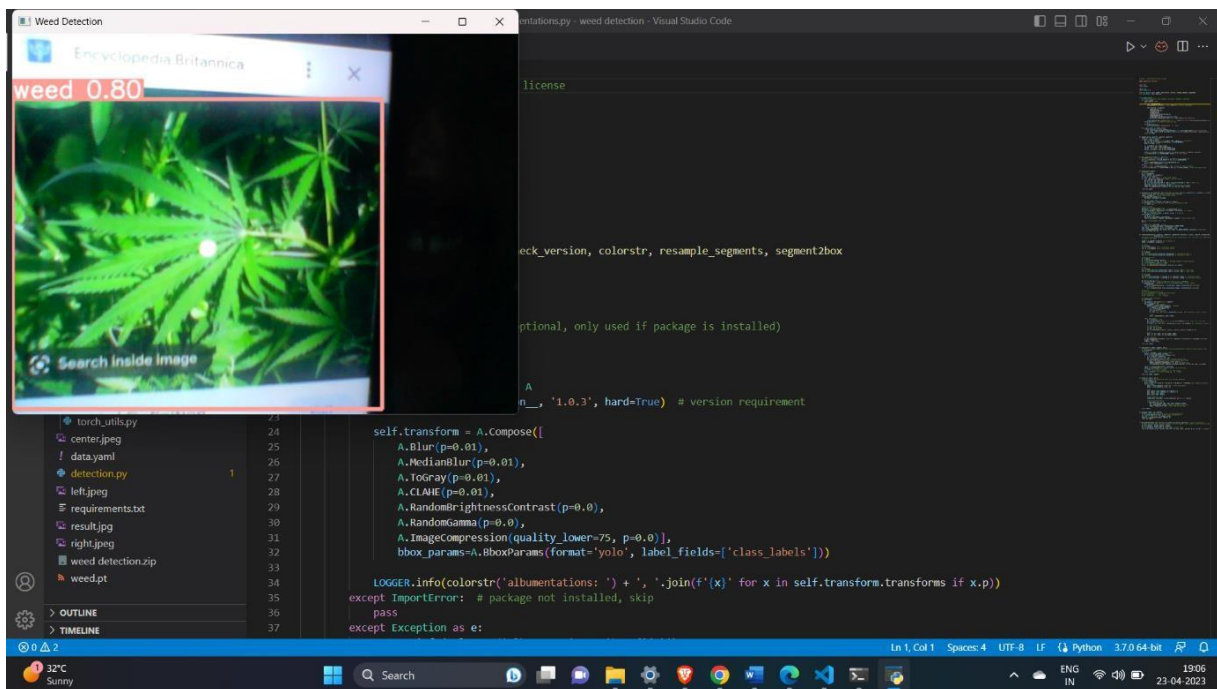


FIG 7.2 WEED DETECTION WITH ITS IMPACT FACTOR

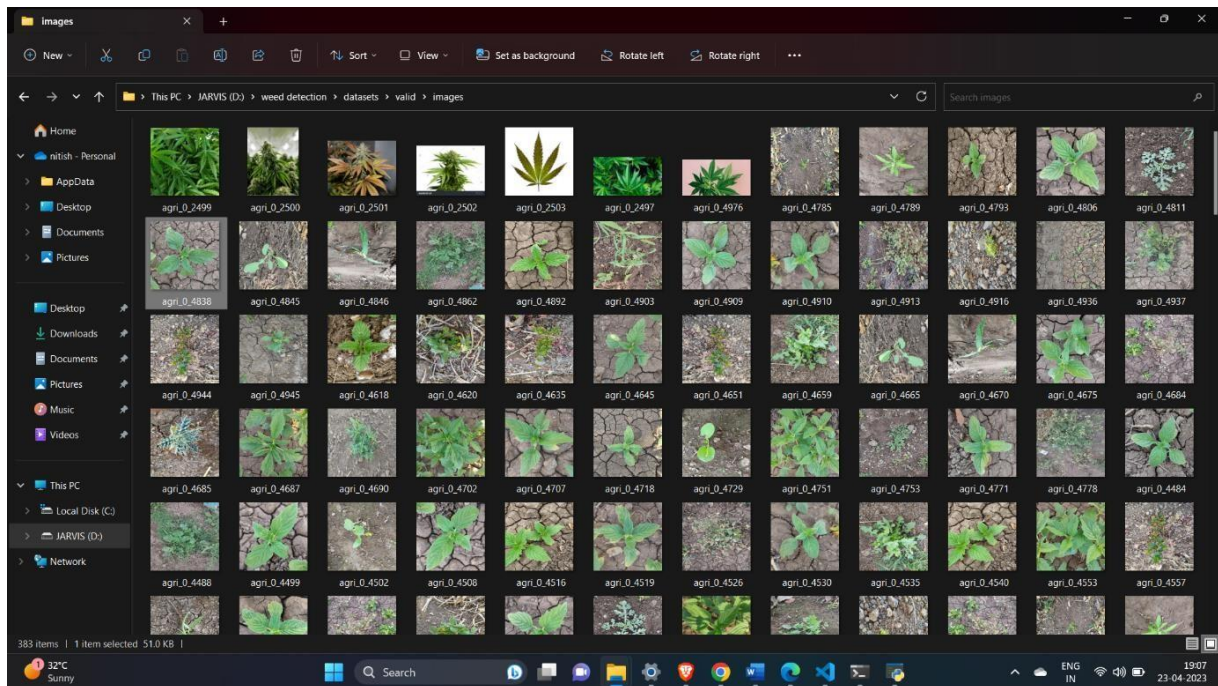


FIG 7.3 WEED COLLECTED DATA SET

CONCLUSION

In conclusion, weed detection is an important aspect of modern agriculture and plays a vital role in ensuring the health and productivity of crops. The development of accurate and efficient weed detection technologies can help farmers to identify and manage weeds in a timely manner, reducing crop losses and improving overall yield. Various techniques and technologies are available for weed detection, including high-resolution imagery, multispectral or hyperspectral imagery, LiDAR data, machine learning algorithms, and mobile applications. These techniques can be combined to create comprehensive datasets for weed detection, which can be used to develop and train algorithms for weed identification and classification. The agriculture image is taken from kaggle database. It consists of 1000 weeds and crops images. 80% of the images are used for training and 20% of the images are used for validation

FUTURE ENHANCEMENT

High-resolution imagery: Using high-resolution imagery, such as aerial or satellite images, can provide a more detailed view of the landscape and help to identify specific weed species. The images can be analyzed using computer vision algorithms to detect the color, shape, and texture of the weeds. Machine learning algorithms: Machine learning algorithms, such as deep learning or support vector machines, can be trained on large datasets of weed and crop images to identify patterns and features that are specific to different weed species. These algorithms can then be used to classify new images and identify the presence of weed. For future developments this proto type can be used for web interface ,robotics, and live weed detection application

REFERENCES

- [1] Ajinkya Paikekari, Vrushali Ghule, et al., Weed detection using image processing International Research Journal of Engineering and Technology (March 2016).
- [2] Riya Desai, et al., Removal of weeds using Image Processing International Journal of Advanced Computer Technology (IJACT) (2016).
- [3] Ashintosh K Shinde, et al., Crop detection by machine vision for weed management International Journal of Advances in Engineering & Technology (July, 2014).
- [4] A Satish kumar, et al., Detection of weeds in a crop row using image processing Imperial Journal of Interdisciplinary Research (2016).
- [5] Amruta A. Aware, Crop and weed detection based on texture and size features and automatic spraying of herbicides International Journal of Advanced Research (2016).
- [6] Mrs. Latha, A Poojith, B V Amarnath Reddy, G Vittal Kumar; “Image Processing In Agriculture”; IJIREEICE; 2014.
- [7] Xavier P. Burgos-Artizzu, Angela Ribeiro, Maria Guijarro, Gonzalo Pajares; “Real- time image processing for crop/weed discrimination in maize fields”; Elsevier; 2010.
- [8] Hossein Nejati, Zohreh Azimifar, Mohsen Zamani; “Using Fast Fourier Transform for weed detection in corn fields”; IEEE; 2008
- [9] Grianggai Samseemoung, Peeyush Soni, Hemantha P. W. Jayasuriya, Vilas M. Salokhe; “Application of low altitude remote sensing (LARS) platform for monitoring crop growth and weed infestation in a soybean plantation”; Springer; 2012